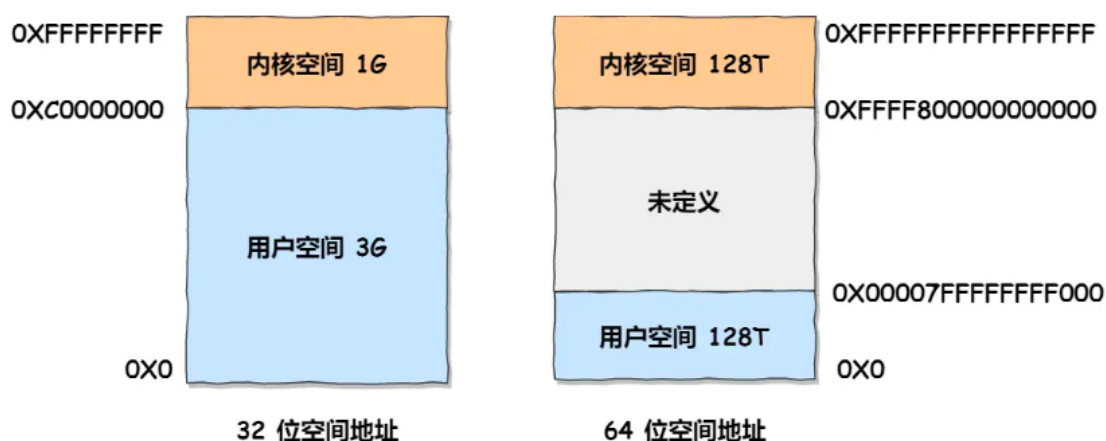


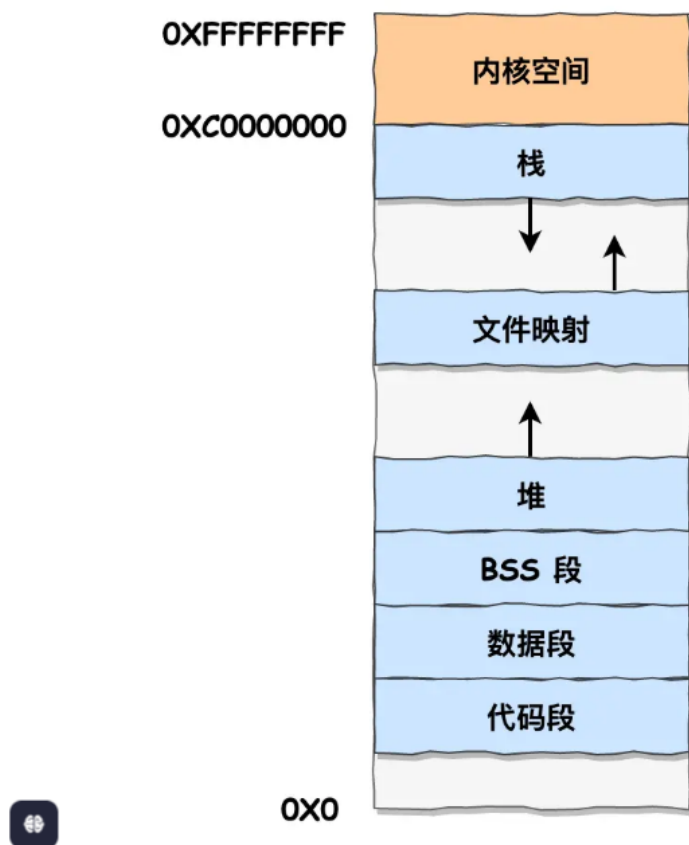
内核空间和用户空间



内核态和用户态，为什么要分这两种态：

内核态权限更高：我们要做某些操作，比如函数调用需要切换到内核态获取权限，用户态权限相对较低

为什么要分：一个是为了安全，防止一直有内核态的权限，进行了一些误操作，另一个是稳定性，因为具有内核态的用户发生崩溃，可能会导致整个操作系统崩溃，所以只有需要用内核权限时才给予分配



用户空间（进程）分布如上：

- 代码段 (text)：可执行二进制代码

- 数据段 (data) : 存储已初始化的全局变量 / 静态变量, 占文件空间和运行时内存空间
- BSS段: 存储未初始化的全局变量 / 静态变量 (初始化为0), 由于初始化为0, 所以不占文件空间, 只占运行时内存空间
- rodata段 (read only) : 只读数据段, **存放一些字符串常量, 虚函数表, 一般放在只读ROM中**
- 堆: 程序员自己分配
- **中间的文件映射, 也叫库: mmap映射的空间就在这里, 还存放动态库, 文件映射等**
- 栈: 函数调用, 局部变量 (一般大小为8M)

3.qt是跨平台的c++应用程序开发框架, 怎么理解跨平台, (你为什么选择qt开发? 你还了解别的图形开发工具吗?)

所谓跨平台, **就是指同一套代码可以在不同操作系统上编译运行, 且功能, 界面保持一致**

为什么选择qt开发 (和别的图形界面开发工具比较) :

- 底层采用c/c++类库 (移植性语言), 能在多平台上部署, 我也了解过一些其他的图形开发工具, 比如Electron基于web技术, 它虽然可以跨平台但依赖Chromium内核 (谷歌), 存在体积大性能较低的问题, 然后像别的MFC/WinForms比较轻便, 在vs上都能配置使用, 但仅仅支持Windows, 无法适配其他系统。而且我们现在像电脑上用的WPS就是基于qt开发的, 别的一些轻量级软件也有用到qt的库 (有哪些库: 网络库QtNetwork.dll 数据库Qt5sql.dll 音频播放Qt5Multimedia.dll等等)

« 本地磁盘 (C:) » Program Files (x86) » Kingsoft » WPS Office » 12.1.0.21915 » office6 »

名称	修改日期	类型	大小
addons	2025/7/5 20:47	文件夹	
cfgs	2025/7/21 18:58	文件夹	
data	2025/7/5 20:47	文件夹	
dicts	2025/7/5 20:47	文件夹	
docs	2025/7/5 20:47	文件夹	
document theme	2025/7/5 20:47	文件夹	
iconthemes	2025/7/5 20:47	文件夹	
kvprinter	2025/7/5 20:47	文件夹	
menuext	2025/7/5 20:49	文件夹	
mui	2025/7/5 20:47	文件夹	
qt	2025/7/5 20:48	文件夹	

- qt具有完善的工具链（本身集代码编辑，调试，UI设计为一体），支持可视化布局 and 实时预览，同时还有完整的社区和生态，文档之类的，遇到了相应的问题可以比较快的找到解决方法，具有优秀的信号槽机制，可以高效地实现类与类之间的通信
- 我开发音视频项目，通常需要对图像进行渲染，或者开硬件加速，而我了解qt恰好支持硬件加速功能，同时他内部还支持有关opengl相应图形渲染库，比如组件里面就有QOpenGLWindow和QOpenGLWidget，对我项目的实现提供了不少便利

4.qt信号和槽机制

信号和槽底层基于元对象机制，具体处理流程如下

- 预处理期处理宏Q_OBJECT
- MOC（Meta-Object Compiler）给对应有宏的类生成 moc_类名.cpp文件（包含一个静态的元对象实例，记录该类中包含的信号和槽信息）
- 在QObject::connect函数调用后，底层调用QMetaObject::connect函数来查找信号和槽，创建其索引，并将它们存储映射表中
- 当信号被发射时，QMetaObject会遍历映射表，找到存储的索引，并调用相应的槽函数。**Qt的信号和槽是线程安全的**，如果信号和槽在不同线程，Qt会自动将槽的调用转为事件，通过事件循环分发给目标线程

5.connect的五个参数

AutoConnection, 默认值, 如果信号发送者和接收者在同一线程, 则用
DirectConnection, 否则用QueueConnection

DirectConnection: 槽函数在信号发送时直接被调用, 这在多线程环境下比较危险

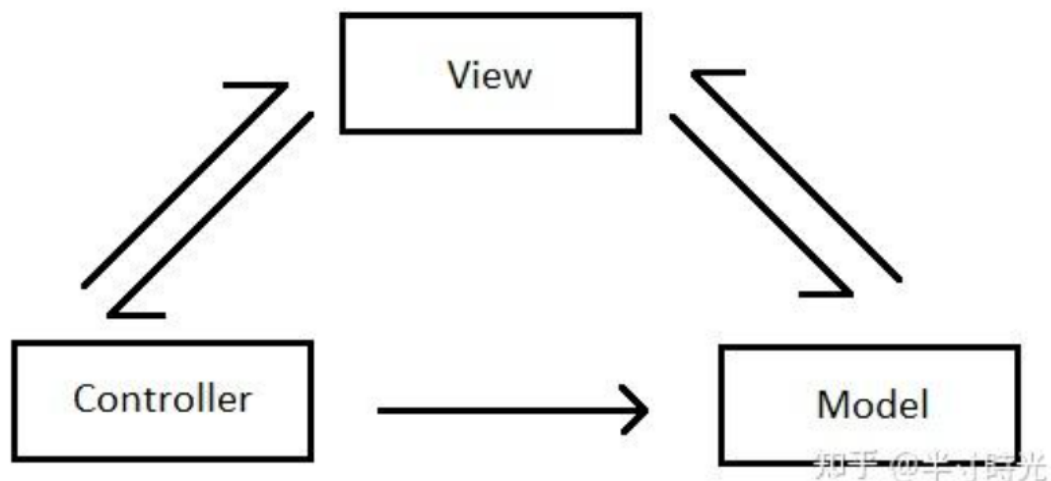
QueueConnection: 槽函数被放入事件循环, 多线程一般使用这个

BlockingQueueConnection: 调用时机和QueueConnction一致, 但是信号发出者发出信号后会等待槽函数执行完再结束, 常用于多线程同步

UniqueConnection: 同样的信号与槽连接只计入一次, 前面几种可以按位或进行结合使用

6.项目中有用到软件架构没, 比如MVC, MVP架构和MVVM架构

MVC架构:



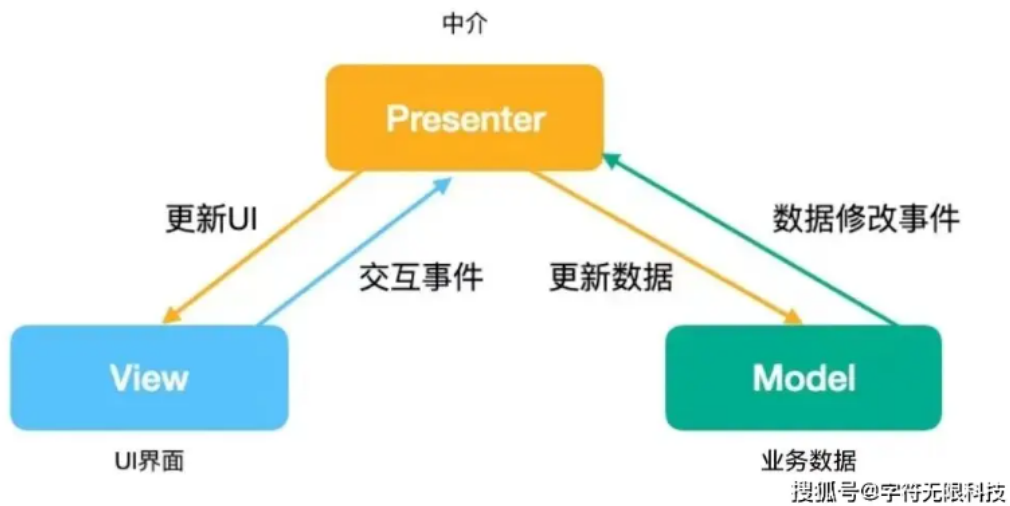
其中模型 (Model) 层负责相应业务逻辑, 比如数据增删改查相关处理

视图 (View) 负责接收用户交互请求并给用户展示数据

控制器 (Controller) 负责视图和模型连接, 接收视图层的请求并交给模型层处理

在我的短视频项目中, 视图层就是qt的ui界面显示。模型层就是视频播放模块, 编码上传视频模块, 进行一个具体的业务处理。控制器层主要就是信号和槽机制, 把用户的页面操作变成具体的模型层需要处理的需求

MVP架构:



MVP架构把和MVC架构最大区别在于视图层的业务大部分交给Presenter层处理, Model层只拿来更新一下数据

MVVM架构



MVVM架构把视图层数据绑定在ViewModel层, 通过监听ViewModel层的属性变化, 调用Model层的接口处理业务并更新数据